

Lista de Atividades

Classes em C++ para Objetos Físicos: MET e Traços Reconstruídos

Marco Dias

22 de agosto de 2026

Instruções gerais de entrega

- A atividade deve ser resolvida em C++ e entregue através do repositório Git de cada aluno (ver instruções de configuração de Git/GitHub fornecidas anteriormente no curso).
- Crie uma pasta para cada exercício (por exemplo `ex1_met/` e `ex2_track/`), contendo os arquivos de cabeçalho (`.h`), implementação (`.cpp`), um `Makefile` e o programa de teste correspondente.
- Cada `Makefile` deve compilar o código em uma *biblioteca compartilhada* (`.so`) e, em seguida, ligar essa biblioteca ao programa de teste.
- Todos os dados-membro das classes devem ser **privados** (ou **protected**, quando estritamente necessário para herança), seguindo o princípio de ocultamento de informação (*encapsulamento*) discutido em aula.
- Faça o `commit` e `push` do código para o seu repositório antes do prazo de entrega. Inclua um pequeno `README.md` explicando como compilar e rodar cada teste.

1 Energia Transversa Faltante (Missing E_T)

1.1 O que é o Missing E_T ?

Em um colisor de hádrons como o LHC, os dois feixes colidem ao longo do eixo z , de modo que o momento total inicial no plano transversal (plano x - y , perpendicular ao eixo dos feixes) é nulo. Pela conservação de momento, a soma vetorial dos momentos transversos de *todas* as partículas produzidas em uma colisão também deve ser nula.

O detector, no entanto, não consegue medir diretamente partículas que interagem muito fracamente com a matéria, como os neutrinos, nem partículas hipotéticas eletricamente neutras e estáveis previstas por modelos além do Modelo Padrão (por exemplo, candidatos a matéria escura). Essas partículas atravessam o detector sem deixar sinal.

Definimos o vetor de **energia transversa faltante** (*Missing Transverse Energy*, $\vec{E}_T^{miss}$, frequentemente abreviado MET) como o negativo da soma vetorial dos momentos transversos de todos os objetos reconstruídos e visíveis no evento (elétrons, múons, fótons, jatos, etc.):

$$\vec{E}_T^{miss} = - \sum_{i \in \text{objetos visíveis}} \vec{p}_{T,i}. \quad (1)$$

O módulo desse vetor, $E_T^{miss} = |\vec{E}_T^{miss}|$, é a quantidade escalar comumente chamada de MET. Um valor grande de MET é um indício de que uma ou mais partículas não detectadas (tipicamente neutrinos) foram produzidas no evento — por isso o MET é uma variável central em análises que envolvem decaimentos com neutrinos (por exemplo $W \rightarrow \ell \nu$) e em buscas por nova física.

Como o MET é obtido por diferença entre grandezas medidas, ele é sensível a qualquer erro de medida, energia mal calibrada, objetos não reconstruídos ou zonas mortas do detector, sendo portanto uma das variáveis mais delicadas de se calcular corretamente em um experimento real.

1.2 Exercício 1 – Classe SimpleMET

Na aula vimos o esqueleto de uma classe para representar o MET de um evento (arquivo SimpleMET.h):

Listing 1: Esqueleto apresentado em aula

```

1 class SimpleMET {
2 public:
3     SimpleMET();
4     ~SimpleMET();
5     void Add(MyTrack);
6     void Add(MyJet);
7     MyMET GetMET();
8     double Value();
9 private:
10    // Membros privados
11 };

```

Sua tarefa é implementar uma versão completa e independente desta ideia. Escreva uma classe em C++, SimpleMET, que representa uma medida de energia transversa faltante (MET). A classe deve:

- Armazenar internamente as componentes cartesianas do vetor MET, E_x^{miss} e E_y^{miss} , como dados-membro **privados** (por exemplo `double mex_, mey_;`).
- Fornecer um construtor que inicialize o vetor MET a zero, e um construtor alternativo que receba E_x^{miss} e E_y^{miss} diretamente.
- Fornecer as seguintes funções-membro públicas (todas marcadas `const`, pois não devem alterar o estado do objeto):

- `double Value() const;` – retorna o módulo $E_T^{miss} = \sqrt{(E_x^{miss})^2 + (E_y^{miss})^2}$.
- `double Ex() const;` e `double Ey() const;` – retornam as componentes cartesianas.
- `double Phi() const;` – retorna o ângulo ϕ entre o vetor MET e o eixo x positivo no plano transversal, isto é, $\phi = \text{atan2}(E_y^{miss}, E_x^{miss})$, no intervalo $(-\pi, \pi]$.

- Fornecer uma função `void Add(double px, double py);` que simula a adição de um objeto reconstruído (por exemplo, um jato ou uma partícula) ao cálculo do MET, subtraindo o momento transversal desse objeto do vetor MET acumulado, conforme a definição da Seção 1.1.

Requisitos de entrega:

- Separe a declaração da classe em `SimpleMET.h` e a implementação em `SimpleMET.cpp`.
- Escreva um `Makefile` que compile `SimpleMET.cpp` em uma biblioteca compartilhada `libSimpleMET.so` (dica: use as flags `-fPIC` e `-shared` do `g++`).
- Escreva um programa de teste `testMET.cpp` que crie um objeto `SimpleMET`, adicione dois ou três objetos com componentes (p_x, p_y) escolhidas por você, e imprima na tela o valor, as componentes e o ϕ do MET resultante, verificando que os resultados fazem sentido fisicamente (por exemplo, que a soma de dois objetos opostos entre si resulta em MET nulo).

- O `Makefile` deve também compilar e ligar `testMET.cpp` contra `libSimpleMET.so`, gerando um executável `testMET`.

2 Traços Reconstruídos (Reconstructed Tracks)

2.1 O que é um traço reconstruído?

Detecores de traçamento (*tracking detectors*), como os detectores de silício e as câmaras de deriva que compõem o sistema interno de um experimento como o CMS ou o ATLAS, registram uma série de pontos (*hits*) deixados pela passagem de partículas carregadas através de camadas sensíveis de material, imersas em um campo magnético intenso.

Um **traço reconstruído** (*reconstructed track*) é o resultado do processo de reconstrução que associa esses *hits* individuais a uma trajetória contínua, ajustando uma curva (tipicamente uma hélice, para uma partícula carregada em um campo magnético uniforme) aos pontos medidos. Desse ajuste obtém-se um conjunto de parâmetros que descrevem completamente a cinemática da partícula naquele ponto, entre eles:

- o momento tridimensional $\vec{p} = (p_x, p_y, p_z)$ da partícula (obtido a partir da curvatura da trajetória no campo magnético, via $p_T \propto 1/R$, onde R é o raio de curvatura);
- a carga elétrica (obtida do sinal da curvatura);
- o ponto de origem do traço (parâmetros de impacto), útil para associar o traço a um vértice primário ou secundário.

A partir do momento, é comum derivar quantidades cinemáticas convenientes para a física de colisores, como o momento transversal p_T (momento no plano perpendicular ao feixe) e a pseudorapidez η , que descreve o ângulo polar da trajetória de forma aproximadamente invariante por *boosts* de Lorentz ao longo do eixo do feixe.

É importante notar que um traço reconstruído é uma quantidade puramente experimental: ele é obtido a partir de medidas do detector e do algoritmo de reconstrução, sem qualquer conhecimento sobre qual tipo de partícula realmente o gerou. Em simulações (Monte Carlo), no entanto, conhece-se a *verdade simulada* (*Monte Carlo truth*): sabe-se exatamente qual partícula gerou aquele traço e de qual partícula-mãe ela se originou, informação usada para validar e calibrar os algoritmos de reconstrução.

2.2 Exercício 2 – Classes Track e SimTrack

- (a) **Classe base Track.** Escreva uma classe em C++, `Track`, que representa um traço reconstruído, com os seguintes requisitos:

- Os dados-membro devem armazenar o quadrimomento da partícula, (E, p_x, p_y, p_z) , como **dados privados** (por exemplo `double e_, px_, py_, pz_;`).
- Um construtor que receba as quatro componentes do quadrimomento.
- Funções-membro públicas `const` para acessar as componentes brutas: `E()`, `Px()`, `Py()`, `Pz()`.
- Uma função `double Pt() const;` que retorne o momento transversal,

$$p_T = \sqrt{p_x^2 + p_y^2}. \quad (2)$$

- Uma função `double Eta() const;` que retorne a pseudorapidez,

$$\eta = -\ln \left[\tan \left(\frac{\theta}{2} \right) \right], \quad \theta = \arccos \left(\frac{p_z}{|\vec{p}|} \right), \quad |\vec{p}| = \sqrt{p_x^2 + p_y^2 + p_z^2}, \quad (3)$$

onde θ é o ângulo polar entre o vetor momento $\vec{p}$ e o eixo z (eixo do feixe). *Dica:* cuidado com o caso $p_T = 0$ (traço puramente ao longo do eixo z), que corresponde a $\eta \rightarrow \pm\infty$; trate esse caso separadamente para evitar divisões por zero ou erros de domínio no $\tan/\log$.

(b) **Classe derivada SimTrack.** Em simulações, conhecemos a verdade simulada associada a cada traço. Escreva uma classe **SimTrack** que **herda publicamente** de **Track** e adiciona:

- um dado-membro **privado** para o identificador da partícula que gerou o traço (`int pdgId_;`, seguindo a convenção de códigos PDG: por exemplo 11 para elétron, 211 para pión carregado);
- um dado-membro **privado** para o identificador da partícula-mãe (`int parentPdgId_;`);
- um construtor que receba o quadrimomento (repassado ao construtor da classe base via lista de inicialização) mais os dois identificadores acima;
- funções-membro públicas `const`, `ParticleId()` e `ParentId()`, para acessar essas informações.

Lembre-se de que, seguindo o princípio de encapsulamento, os dados devem permanecer inacessíveis diretamente de fora da classe – todo o acesso deve ocorrer através das funções-membro públicas.

Requisitos de entrega:

- Separe declaração e implementação: `Track.h/Track.cpp` e `SimTrack.h/SimTrack.cpp` (este último incluindo `Track.h`).
- Escreva um único `Makefile` que compile as duas classes em bibliotecas compartilhadas (`libTrack.so` e `libSimTrack.so`), tomando cuidado para ligar `libSimTrack.so` contra `libTrack.so`.
- Escreva um programa de teste `testTrack.cpp` que:
 1. crie ao menos dois objetos **SimTrack** com quadrimomentos diferentes (escolhidos por você, respeitando $E^2 = p_x^2 + p_y^2 + p_z^2 + m^2$ aproximadamente, ou simplesmente $E \geq |\vec{p}|$);
 2. imprima p_T , η , o identificador da partícula e o identificador da partícula-mãe de cada traço;
 3. inclua pelo menos um caso com p_T pequeno/próximo de zero para verificar que o tratamento do caso limite em `Eta()` funciona corretamente.
- O `Makefile` deve compilar e ligar `testTrack.cpp` contra as duas bibliotecas, gerando o executável `testTrack`.

Sugestão de estrutura de diretórios

```
f056-atividade/
+-- ex1_met/
|   +-- SimpleMET.h
|   +-- SimpleMET.cpp
|   +-- testMET.cpp
|   +-- Makefile
+-- ex2_track/
|   +-- Track.h
|   +-- Track.cpp
|   +-- SimTrack.h
```

```
|   +-- SimTrack.cpp  
|   +-- testTrack.cpp  
|   +-- Makefile  
+-- README.md
```

Dica geral de compilação de bibliotecas compartilhadas com g++:

```
g++ -fPIC -c SimpleMET.cpp -o SimpleMET.o  
g++ -shared -o libSimpleMET.so SimpleMET.o  
g++ testMET.cpp -L. -lSimpleMET -Wl,-rpath,. -o testMET
```